Union and Intersection in SKJS: an Audit of the Lattice Operations

Deep Dive B for "Refinement and Inference Growth in SKJS"

Models revision `cf5a6bc7` of `main` (2026-07-09)

Abstract

The main soundness paper assumes, as its rows **L-UNION** and **L-INTER**, that `TType::union` over-approximates set union, that `TType::inter` over-approximates set intersection, and — the load-bearing extra used by every branch-dropping refinement rule — that an intersection result of $\bot$ implies semantic disjointness. This deep dive audits both operations level by level against the pinned revision. The outcome is asymmetric. Union is *proved*: every stratum over-approximates, and all strata that membership semantics can observe are in fact exact; the `seenUnion` cache is unconditionally sound because the union computation never reads the substitution. Intersection is *refuted as stated* by one defect that makes `inter` return $\bot$ on semantically overlapping operands — the treatment of a failed call/new signature intersection as branch disjointness — with a second, string-kind defect inheriting the **L-LIT**(ii) escape-spelling violation. Both are empirically witnessed at the pinned revision: the checker silently narrows an inhabited variable to `never` in a reachable branch. Every other intersection stratum (units, tags, named fields, indexes under the covering semantics, the array brand, the per-tag `STypes` assembly, the instance layer) is proved sound, so the ledger row splits into proved strata, the two violated strata, and one named residual obligation (the `seenInter` cache under candidate freezing).

Contents

1 Purpose and Ledger Rows Addressed **2**

2 Recap: Semantics and the Shape of the Two Operations **2**

3 Union, Level by Level **3**
- 3.1 Literal sets and primitive members . . . 3
- 3.2 Object branches: set union . . . 4
- 3.3 STypes, Clause, Clauses, TInsts . . . 4
- 3.4 The union theorem . . . 5
- 3.5 The branch-merging client: TFields::union . . . 5

4 Intersection, Level by Level **6**
- 4.1 Literal sets and primitive members . . . 6
- 4.2 The per-tag assembly: STypes::inter . . . 7
- 4.3 Object branch pairs . . . 7
- 4.4 Clause and Clauses intersection . . . 9

4.5 The instance layer: TVar constraints, TInter, speculative returns 9
4.6 The bottom inventory 10

5 The Caches: seenUnion and seenInter **11**
5.1 seenInter: the sealing argument and its freeze exception 12

6 Empirical Witnesses **13**
6.1 Probe 2: signature bottom (Finding 13) 13
6.2 Probe 3: literal spellings (Finding 19) 13
6.3 Probe 4: type-level intersections 14
6.4 Failed proof attempts, recorded 14

7 Status and Ledger Edits **14**
7.1 Status by stratum 14
7.2 Stratum outcomes 15

8 Implementation Anchor Index **16**

1 Purpose and Ledger Rows Addressed

This document is deep dive B in the series supporting the main paper (*Refinement and Inference Growth in SKJS*, same pinned revision). It addresses exactly two rows of the main paper's "Assumptions and Proof Obligations" ledger: **L-UNION** (union over-approximates set union at all levels), established here *exact at every stratum*; and **L-INTER** (intersection over-approximates, with the $\bot$-implies-disjointness clause), which splits here into an exact containment and the $\bot$-clause, refuted in two strata.

Note that the two halves of L-INTER are one property: if $\mathsf{inter}(t_1, t_2) = \bot$ while $[\![t_1]\!]_\rho \cap [\![t_2]\!]_\rho \neq \emptyset$, then the containment fails at that pair (nothing is in $[\![\bot]\!] = \emptyset$), and conversely. The "$\bot$ implies disjointness" phrasing is called out separately in the main paper only because it is the form its refinement rules consume: R-Eq's true branch installs $\mathsf{inter}(\Gamma(x), \tau_e)$ directly, and R-EqField, R-Inst, and R-IsArray *drop an object branch* (or the whole variable, for R-Eq on a single-member type) exactly when an intersection comes back $\bot$. Section 6 exhibits reachable programs where that drop is wrong. The audit proceeds bottom-up through the representation strata of the main paper's Section 3; the precise ledger edits are in Section 7.

impl: `TTypeUnion.sk; TTypeInter.sk; TType.sk / STypes, Clause, Clauses, TFields, PrimLiterals.sk`

2 Recap: Semantics and the Shape of the Two Operations

We reuse the main paper's runtime model and denotations without change: values carry one of nine tags (rtag); simple types are units, literal or full primitives, and object types with branch sets; $[\![\cdot]\!]$ is the literal-closure semantics for primitives and the extensible, get-only membership semantics for object branches (main paper, Section "Internal Type Representation"). A TType is a pair (I, C) of an instance layer $I \in \mathsf{TInsts}$ and clauses $C \in \mathsf{Clauses}$ in DNF, with $[\![(I, C)]\!]_\rho = [\![I]\!]_\rho \cup [\![C]\!]_\rho$; a clause $\langle V \mid S \rangle$ denotes $\bigcap_{v \in V} [\![v]\!]_\rho \cap [\![S]\!]_\rho$ with $S \in \mathsf{STypes} \cup \{\top\}$, and STypes holds at most one SType per tag. All denotations are indexed by the substitution state ρ; we suppress ρ when an argument is pointwise in it.

Union's shape. `TType::union` short-circuits top/bottom, then works *structurally*: instance sets are set-unioned unexpanded, and clauses are merged by var-set key, colliding clauses merging their STypes per tag (literal-set union for primitives, branch-set union for objects). Crucially, no arm of this computation expands an instance or reads a variable binding: union is a pure function of its operands' syntax.
impl: `TTypeUnion.sk / TType::union_, Clauses::union, STypes::union, TObject::union, TInsts::union`

Intersection's shape. `TType::inter` short-circuits top/bottom, then handles bare variables specially (constraint attachment or an unexpanded `TInter`), then *expands both operands to clauses* under the current state (sealing variables it reads covariantly), runs two speculative boxed-primitive subtype checks, and finally distributes: pairwise clause intersection, per-tag STypes intersection, pairwise object-branch intersection, field/index/call/new/brand intersection inside a branch pair. Bottom can be produced at four depths: literal sets, the per-tag map, a branch pair, and the whole clause product.
impl: `TTypeInter.sk / TType::inter_, Clauses::inter, STypes::inter, TObject::inter, TFields::inter`

Method. For each stratum we state the containment against the semantics of the main paper, prove it or reduce it to a named assumption, and separately classify every $\bot$-producing site as *semantic* (disjointness holds) or *heuristic* (it does not). Claims about observable behavior were tested against the pinned checker binary (`target/host/dev/skjs` built from `cf5a6bc7`, run via `make check-file-in-docker`); Section 6 reproduces the probe programs.

3 Union, Level by Level

3.1 Literal sets and primitive members

Lemma 1 (Literal-Set Union is Exact)**.** *For literal sets L_1, L_2 of the same kind k: $[\![\mathsf{prim}(k, L_1)]\!] \cup [\![\mathsf{prim}(k, L_2)]\!] = [\![\mathsf{prim}(k, L_1 \cup L_2)]\!]$, where $L_1 \cup L_2$ is the implementation's union of spelling sets.*

Proof. By the closure semantics (main paper, Definition of literal denotation), $[\![\mathsf{prim}(k, L)]\!] = \bigcup_{\ell \in L} [\![\mathsf{prim}(k, \{\ell\})]\!]$; union of spelling sets commutes with the outer union. No normalization or injectivity assumption is needed: even $\approx$-equivalent duplicate spellings only make the two sides equal unions of equal parts.
impl: `PrimLiterals.sk / union (keyed set union of spellings)`

□

Lemma 2 (TPrimValue and TPrimUnit Union are Exact)**.** *`TPrimValue::union` on same-kind operands returns: the full primitive if either side is full ($L = \top$ absorbs); otherwise the literal-set union, except that a boolean literal set reaching two elements widens to full `boolean`. Each case is semantically exact. Same-tag units are identical, so `STypes::union`'s unit arm (return the left operand) is exact.*

Proof. Full absorbs because $[\![\mathsf{prim}(k, \top)]\!] \supseteq [\![\mathsf{prim}(k, L)]\!]$ for any L (every literal denotes same-tag values). The literal case is Lemma 1. For boolean: the only spellings of kind `bool` are `true` and `false`, so a two-element set already denotes all of $\{(\mathsf{bool}, \mathsf{true}), (\mathsf{bool}, \mathsf{false})\}$; the widening is an equality. Units: a tag determines its unit, and `mapUnion` pairs members of identical tags only.
impl: `TTypeUnion.sk / TPrimValue::union; STypes::union (TPrimUnit arm)`

□

3.2 Object branches: set union

Lemma 3 (Object Union is Exact)**.** $[\![\mathsf{object}(B_1)]\!] \cup [\![\mathsf{object}(B_2)]\!] = [\![\mathsf{object}(B_1 \cup B_2)]\!]$.

Proof. Membership in $\mathsf{object}(B)$ is an existential over branches (main paper, Definition of object-type semantics), and existentials distribute over set union of the branch set. `TObject::union` is exactly $B_1 \cup B_2$; the `unexpandedType` hint is reset to `None`, which is irrelevant to $[\![\cdot]\!]$ (the main paper excludes it from semantics). No branch is merged, so no precision or soundness question arises at this level.
impl: `TTypeUnion.sk / TObject::union (branches.union)`

□

The nontrivial object-*merging* code (`TFields::union`, `TField::union`) is *not* on the $\mathsf{union}(t_1, t_2)$ path: it is a separate client used to collapse a multi-branch object into a single summary branch for field reads. We audit it in Section 3.5 because the main paper's R-EqField proof leans on the resolved read type it produces, and L-UNION is stated "at all levels".

3.3 STypes, Clause, Clauses, TInsts

Lemma 4 (STypes Union is Exact)**.** $[\![S_1]\!] \cup [\![S_2]\!] = [\![\mathsf{STypes::union}(S_1, S_2)]\!]$.

Proof. `mapUnion` keeps members whose tag appears on one side and merges same-tag pairs with Lemmas 2 and 3. Since $[\![S]\!] = \bigcup_{s \in S} [\![s]\!]$ and tags partition Val (main paper, Lemma A0/A1), grouping the union by tag is exact, and each per-tag merge is exact.
impl: `TTypeUnion.sk / STypes::union; InternalKeyedArrayMap.sk / mapUnion`

□

Lemma 5 (Clause Collision Merge is Exact)**.** *For clauses with equal var-sets V: $[\![\langle V \mid S_1 \rangle]\!] \cup [\![\langle V \mid S_2 \rangle]\!] = [\![\mathsf{Clause::union}]\!]$, where the merge yields $\langle V \mid \top \rangle$ if either operand's STypes is $\top$ and $\langle V \mid \mathsf{STypes::union}(S_1, S_2) \rangle$ otherwise.*

Proof. Writing $A = \bigcap_{v \in V} [\![v]\!]$, the left side is $(A \cap [\![S_1]\!]) \cup (A \cap [\![S_2]\!]) = A \cap ([\![S_1]\!] \cup [\![S_2]\!])$ by distributivity of $\cap$ over $\cup$; then Lemma 4, or $[\![S_i]\!] = \mathsf{Val}$ absorbing when a side is $\top$. This is the "key-collision widening" of the operand pair: it is not a widening at all semantically — the collision merge is exact.
impl: `TTypeUnion.sk / Clause::union`

□

Lemma 6 (Clauses Union is Exact; Top-Collapse is Sound)**.** $[\![C_1]\!] \cup [\![C_2]\!] = [\![\mathsf{Clauses::union}(C_1, C_2)]\!]$, *and the constructor invariant (a top clause, if present, is alone) is preserved.*

Proof. $[\![C]\!] = \bigcup_{c \in C} [\![c]\!]$; `mapUnion` keyed by var-set keeps unmatched clauses and merges matched ones by Lemma 5, so the result denotes the union of all clause denotations on both sides. `Clauses::create` then collapses to $\{\langle \emptyset \mid \top \rangle\}$ if any clause is top; this replaces the set with a superset ($[\![\langle \emptyset \mid \top \rangle]\!] = \mathsf{Val}$), preserving the containment (and losing exactness only in the case where the union already contained Val, so not even there).
impl: `TTypeUnion.sk / Clauses::union; TType.sk / Clauses::create`

□

Lemma 7 (Instance-Layer Union is Exact)**.** $[\![I_1]\!]_\rho \cup [\![I_2]\!]_\rho = [\![I_1 \cup I_2]\!]_\rho$ *at every state ρ.*

Proof. $[\![I]\!]_\rho = \bigcup_{i \in I} [\![i]\!]_\rho$ and `TInsts::union` is set union of unexpanded instances; no instance is rewritten, so the identity holds pointwise in ρ. This is the design point the implementation comments call out: union keeps variables and named instances side by side rather than resolving them.
impl: `TTypeUnion.sk / TInsts::union`

□

3.4 The union theorem

Theorem 8 (L-UNION, Clause and Instance Strata)**.** *For all t_1, t_2 and every state ρ: $[\![t_1]\!]_\rho \cup [\![t_2]\!]_\rho \subseteq [\![\mathsf{union}(t_1, t_2)]\!]_\rho$. Moreover the containment is an equality except possibly at the top/bottom short-circuits (where it is an equality too: $t \vee \top = \top$, $t \vee \bot = t$) — i.e. union is semantically exact for the membership semantics.*

Proof. If either operand is top or bottom, the short-circuit returns the exact answer. Otherwise $\mathsf{union}(t_1, t_2) = (I_1 \cup I_2,\ \mathsf{Clauses::union}(C_1, C_2))$ and $[\![(I, C)]\!] = [\![I]\!] \cup [\![C]\!]$ splits the claim into Lemmas 7 and 6. The equality cache check `ty1 == ty2` returns `ty1`, exact. Since every step is either a pure set operation or one of the exact merges above, and none reads ρ, the result holds at every state uniformly — a fact we reuse for the cache in Section 5.
impl: `TTypeUnion.sk / TType::union, union_`

□

Two remarks. First, exactness is relative to the main paper's *membership* semantics, in which function signatures are opaque; the signature bodies that `TFun::union` builds (parameters intersected contravariantly, returns unioned) never enter $[\![\cdot]\!]$ and are part of the subtyping surface instead — and `TFun::union` is not reachable from $\mathsf{union}(t_1, t_2)$ anyway (only from the branch-merging client below). Second, `unionN` of an empty sequence returns $\bot$, the exact empty union.

3.5 The branch-merging client: TFields::union

A multi-branch object's field reads union the per-field types across the branches: `maybeGetType` combines each branch's field with `TField::union` (via `TType::unionN`), and the sibling read paths (`resolveLiteralAccess`, `maybeGetSymbol`, `maybeGetCall`/`maybeGetNew`, `maybeGetNonSyntheticIndexForKeyKi
do the same; these feed the resolved read types of R-EqField. The correctness statement is branch-level:

Lemma 9 (Branch Merge Over-Approximates)**.** *For object branches b_1, b_2: $[\![b_1]\!] \cup [\![b_2]\!] \subseteq [\![\mathsf{TFields::union}(b_1, b_2)]\!]$.*

Proof. Let $o \in [\![b_1]\!]$ (the other case is symmetric); we check every constraint of the merged branch b. *Named fields.* A field in both sides merges with get-union and $\mathsf{opt}_1 \vee \mathsf{opt}_2$: if o has the key, its value is in $\mathsf{get}_1 \subseteq$ the union (Theorem 8); if absent, the field is required only if required on both sides, contradicting $o \in [\![b_1]\!]$ when required there — so optionality is respected. A field only in b_1: if b_2 has an index covering its key, the merge keeps it unioned with the index-as-named field, which is *optional* (indexes carry $\mathsf{opt} = \mathsf{true}$), so the disjunct $o \in [\![b_2]\!]$ also satisfies it; if b_2 has no covering index, the field is *dropped*, sound under the extensible semantics. *Indexes.* A per-kind index survives only if both sides have one covering that kind (the number index may be covered by a string index, per the covering semantics), with get-union. Take a prop p of o covered by the merged index and not named in b. Names in b include every b_1 name that survived; a b_1 name is dropped only when b_2 lacks a covering index — but then the merged index of that kind would

not exist either. So p is covered-and-not-named in b_1 as well, and $\mathsf{get}(o,p) \in [\![\mathsf{idx}_1.\mathsf{get}]\!] \subseteq$ the union. The final adjustment $\mathsf{idx}_{\mathsf{num}} := \mathsf{idx}_{\mathsf{num}} \sqcap \mathsf{idx}_{\mathsf{str}}$ (maintaining the invariant $N.\mathsf{get} <: S.\mathsf{get}$) is sound because a numeric-kind prop of o is constrained by *both* the effective number index and the string index of b_1 (a string index covers numeric keys), so its value is in both gets, hence in their intersection — *this* appeal is to intersection soundness at the field-type level and is unconditional there (Section 4.3). *Call/new.* One side absent yields mode Maybe, which constrains nothing; both present yields $\mathsf{opt}_1 \vee \mathsf{opt}_2$ over the signature union, and only the Definite/None modes are semantically visible: o callable per b_1's Definite stays callable; the merged mode is Definite only if both were. Signatures are opaque. *Brand.* Kept only when the two sides agree; otherwise None (unconstrained), a widening. *Set-types and access.* Membership never reads set (get-only semantics) — the merge intersects set-types, the correct dual for write-through-the-summary, but nothing here depends on it; and `FieldAccess::combine` is most-restrictive, which only *restricts* programs (main paper, Definition of object semantics, Never/Phantom discussion).
impl: `TType.sk / maybeGetType, resolveLiteralAccess, maybeGetSymbol, maybeGetCall, maybeGetNew, maybeGetNonSyntheticIndexForKeyKind; TTypeUnion.sk / TField::union, unionIndex`

□

Unlike the branch-set union of Lemma 3, the merge is a genuine over-approximation (field gets are unioned across branches), which is why the implementation uses it only for reads, never as the representation of a union type.

4 Intersection, Level by Level

We now audit `TType::inter` in the same bottom-up order. The summary: the primitive stratum is exact *modulo spelling injectivity* (**L-LIT**(ii), known violated); the per-tag assembly in `STypes::inter` has an outright bug; inside an object branch pair, fields, indexes and the brand are sound (with the brand and the no-shared-tag cases being the only *semantic* bottoms), but the call/new arm converts a failed signature intersection into a branch drop, which is heuristic. The instance layer is sound. Throughout, `inter_` expands its operands as it runs (covariant reads seal variables), an evolution $\rho \to \rho'$; the operands transport across it by **L-EXP** (expansion preserves or enlarges semantics; imported from deep dive C), and the containments below, being pointwise in the state, hold at ρ'.

4.1 Literal sets and primitive members

Lemma 10 (Literal-Set Intersection is Exact Modulo L-LIT(ii))**.** *Assume distinct spellings in $L_1 \cup L_2$ denote $\approx$-inequivalent values (**L-LIT**(ii)). Then $[\![\mathsf{prim}(k, L_1)]\!] \cap [\![\mathsf{prim}(k, L_2)]\!] = [\![\mathsf{prim}(k, L_1 \cap L_2)]\!]$, with $\bot$ (`None`) when the spelling intersection is empty; under the assumption that bottom is semantic. Without the assumption, both the containment and the disjointness reading fail (Finding 19).*

Proof. ($\supseteq$ direction of the equality is immediate from $L_1 \cap L_2 \subseteq L_i$.) For $\subseteq$: let $v \approx \delta_k(\ell_1) \approx \delta_k(\ell_2)$ with $\ell_i \in L_i$ (v is in its own $\approx$-domain since literal denotations exclude NaN). By transitivity $\delta_k(\ell_1) \approx \delta_k(\ell_2)$, so by injectivity $\ell_1 = \ell_2 \in L_1 \cap L_2$. The implementation intersects *spelling* sets, so this is exactly its computation. If injectivity fails — as for the string kind, whose escape spellings are stored undecoded — two spellings of the same value make $L_1 \cap L_2 = \emptyset$ while the semantic intersection is inhabited.
impl: `PrimLiterals.sk / inter; TTypeInter.sk / TPrimValue::inter`

□

Lemma 11 (TPrimValue/TPrimUnit Intersection)**.** *On the* `STypes::inter` *path operands always have equal kinds (the map is tag-keyed), and: full absorbs ($\top \cap L = L$, exact); two literal sets reduce to Lemma 10; same-tag units are equal, and returning the left one is exact. The kind-mismatch* `None` *arm is unreachable from* `STypes::inter` *and, where reachable (the* `overlaps` *client), is semantic by tag disjointness (A1).*
impl: `TTypeInter.sk / TPrimValue::inter, STypes::inter`

4.2 The per-tag assembly: STypes::inter

The correct per-tag decomposition of an intersection of unions is

$$[\![S_1]\!] \cap [\![S_2]\!] = \bigcup_{k \in \mathsf{Tags}} [\![S_1[k]]\!] \cap [\![S_2[k]]\!],$$

by tag disjointness (A1): tags present on only one side contribute nothing, and each shared tag contributes its per-tag intersection. The sound and exact assembly is therefore: intersect per shared tag, *drop* the tags whose per-tag intersection is empty, and return $\bot$ iff *no* tag survives.

The implementation realises exactly this. `STypes::inter` builds the result with `mapInterOption`, whose combiner returns `Some(f(x,y))` on a shared tag: an empty per-tag intersection (`f` returns `None`) is *dropped* from the map, not treated as the zero of the merge, and unmatched tags are dropped likewise; the wrapping `if (!stypes.isEmpty())` guard then returns $\bot$ exactly when no tag survives. Hence $\mathsf{STypes{::}inter}(S_1, S_2) = \bot$ iff every shared tag has an empty per-tag intersection, i.e. iff $[\![S_1]\!] \cap [\![S_2]\!] = \emptyset$: the assembly is exact and its bottoms are semantic. For example {str:"a", num:{0}} ∩ {str:"a", num:{1}} drops the empty `num` tag and keeps `str:"a"`, so `("a" | 0) & ("a" | 1) = "a"`, not `never`.
impl: `TTypeInter.sk / STypes::inter; InternalKeyedArrayMap.sk / mapInterOption`

With the assembly exact, the stratum's soundness reduces to the per-tag lemmas (11 for primitives, Section 4.3 for objects).

4.3 Object branch pairs

`TObject::inter` forms all pairwise branch intersections and returns $\bot$ iff `TFields::inter` is `None` on every pair. This outer shape is exactly right:

Lemma 12 (Pairwise Branch Product)**.** *If for all $b_1 \in B_1, b_2 \in B_2$ the branch-pair operation satisfies $[\![b_1]\!] \cap [\![b_2]\!] \subseteq [\![\mathsf{inter}(b_1, b_2)]\!]$ (reading $[\![$*`None`*$]\!] = \emptyset$), then $[\![\mathsf{object}(B_1)]\!] \cap [\![\mathsf{object}(B_2)]\!] \subseteq [\![\mathsf{TObject{::}inter}]\!]$, and the $\bot$ result is semantic iff every pair's $\bot$ is.*

Proof. o in both objects picks b_1, b_2 with $o \in [\![b_1]\!] \cap [\![b_2]\!]$; the surviving pair (or the contradiction with its dropped status) is immediate, since the result's branch set contains $\mathsf{inter}(b_1, b_2)$ whenever it is not `None`.
impl: `TTypeInter.sk / TObject::inter`

□

So the object stratum reduces entirely to one branch pair, audited constraint by constraint against the main paper's object semantics. Fix $o \in [\![b_1]\!] \cap [\![b_2]\!]$ and let b be the merged branch.

Brand (sound; semantic bottom). Conflicting brands Some(true) vs Some(false) return `None`: semantic, since $\mathsf{isArrayV}(o)$ cannot be both. Otherwise the known side (if any) is kept, which o

satisfies. The same argument covers the refinement helper `TFields::interIsArray` used by R-IsArray: setting an unknown brand constrains o only per the predicate already established, and its mismatch bottom is semantic.
impl: `TTypeInter.sk / TFields::inter (isArray arms), interIsArray`

Named fields (sound). The field maps merge with: both sides present → `TField::inter` (get intersected — or deferred as an unexpanded intersection when `expandInter` is off, which denotes the same intersection by the `TInter` interpretation — set unioned, $\mathsf{opt}_1 \wedge \mathsf{opt}_2$, access combined); present on one side → kept, intersected with the other side's covering index if one exists (`interIndex`: get intersected, optionality preserved since indexes are optional). Soundness for o: a key required on either side is present (o satisfies that side), so $\mathsf{opt}_1 \wedge \mathsf{opt}_2$ is respected; when the key is present, its value lies in each side's applicable get-constraint — the named field's get on the side that names it, the index's get on a side that covers it without naming it (the index clause of the object semantics applies precisely because the key is *not named there*) — hence in the intersection of the collected gets, recursively by L-INTER at the field-type level. Access combination is most-restrictive (`FAPhantom` absorbs into phantom, distinct-owner privates give `FANever`); phantoms weaken the value constraint to presence-only, a widening, and `FANever` remains an ordinary membership constraint — both restrict-only at access sites, per the main paper. Symbol-keyed fields are the same argument against the symbol index.
impl: `TTypeInter.sk / TFields::interNamedFields, TField::inter, TField::interIndex; TType.sk / FieldAccess::combine`

The set direction (membership-irrelevant; flagged). `TField::inter` *unions* the set-types. For the get-only membership semantics this is vacuous — $[\![b]\!]$ never reads set — so L-INTER is unaffected; that is the whole proof. The choice is the one that makes the merged field a subtype of both operands under setter contravariance (the merged branch must accept every write each operand accepts). But as a *write-through* interface it is the permissive direction: a write accepted because only b_1's set-type allows it can break b_2's get-assumption on the same key. That is a field-write concern — membership is snapshot (main paper, object semantics), writes are governed by subtyping — so we do not count it against L-INTER; we flag it so the eventual mutation model audits `set`-union deliberately rather than inheriting it silently.
impl: `TTypeInter.sk / TField::inter (set => TType::union)`

Indexes (sound, via the covering semantics). Per kind, one side absent keeps the other (an absent index is the unconstrained index, and intersecting constraints keeps the stronger side); both present intersect gets. Then the number index is replaced by $\mathsf{idx}_{\mathsf{num}} \sqcap \mathsf{idx}_{\mathsf{str}}$ when both survive. Soundness for o: take a prop p of o covered by a merged index and not named in b. Since b's names are the union of the sides' names, p is not named on either side, so each side's covering index (string indexes covering numeric-kind keys as in the main paper's covering semantics) constrains $\mathsf{get}(o, p)$; the merged get is the intersection of exactly those constraints — including the $N \sqcap S$ adjustment, since a numeric-kind key not named anywhere is covered by *both* the effective number index and the string index of each side that has them. No index arm returns bottom.
impl: `TTypeInter.sk / TIndexes::inter, TFields::interOptIndex`

Call/new (unsound bottom). One side absent keeps the other: sound — if b_1 has $\mathsf{call} = \mathsf{None}$ (o not callable) and b_2 has a mode, the kept mode is satisfied vacuously (Maybe) or the pair was semantically disjoint anyway ($\mathsf{Definite}$; any result over-approximates $\emptyset$, though $\bot$ would have been more precise). Both present: the optionality conjunction $\mathsf{opt}_1 \wedge \mathsf{opt}_2$ is sound (o callable

whenever either side demands it, and Definite results demand only what some side demanded). The defect is `TFuns::inter`: it intersects the signatures by trying `TFun::isSub` in both directions and returning the smaller side, and when *neither* signature subsumes the other it returns `None` — which `TFields::inter` converts into dropping the whole branch pair. "Neither is a subtype of the other" is not disjointness: under the membership semantics signatures are opaque, and even behaviorally, one function value can satisfy two incomparable signatures (any overload; e.g. a `(x: unknown) => void` value inhabits both `(s: string) => void` and `(n: number) => void`).

Finding 13 (Signature Bottom: L-INTER is Violated at the Call/New Stratum)**.** inter *of two single-branch function types with incomparable signatures is* $\bot$ *although their semantic intersection is inhabited. Empirically confirmed (probe 2, Section 6): in* `if (k === g)` *with* `k: (s: string) => void` *and* `g: (n: number) => void`*, the checker narrows* `k` *to* `never` *and accepts* `const z: never = k`*; no no-overlap warning is emitted (the* `overlaps` *helper itself correctly says reference types always overlap, in direct tension with* `TFun::inter`*). The same conversion makes the type-level* `F = ((s: string) => void) & ((n: number) => void)` *expand to* `never` *(probe 4), so nothing is assignable to* `F` *(spurious rejection — the same defect surfacing as incompleteness). The sound repair is to keep the branch with an overload-like or widened signature (even call mode* Definite *with either signature is membership-sound), never to drop it.*
impl: `TTypeInter.sk / TFun::inter, TFuns::inter, TFields::inter (call/new arms); SType::overlaps (contrast)`

4.4 Clause and Clauses intersection

Lemma 14 (Clause Pair)**.** $[\![\langle V_1 \mid S_1\rangle]\!] \cap [\![\langle V_2 \mid S_2\rangle]\!] \subseteq [\![\langle V_1 \cup V_2 \mid S_1 \sqcap S_2\rangle]\!]$ *(with* $\top$ *absorbing on either side, kept exactly),* given *the STypes-level containment; the* `None` *result inherits its status from* `STypes::inter`*, which is exact (Section 4.2), so the clause-level bottom is semantic.*

Proof. $(\bigcap_{V_1} \cap [\![S_1]\!]) \cap (\bigcap_{V_2} \cap [\![S_2]\!]) = \bigcap_{V_1 \cup V_2} \cap ([\![S_1]\!] \cap [\![S_2]\!])$, then the STypes containment. The $\top$ arms are the identities $\mathsf{Val} \cap X = X$.
impl: `TTypeInter.sk / Clause::inter`

□

Lemma 15 (Clause Product and Collision Re-Union)**.** $[\![C_1]\!] \cap [\![C_2]\!] \subseteq [\![\mathsf{Clauses::inter}(C_1, C_2)]\!]$ *given Lemma 14's hypothesis, and the constructor invariant is preserved.*

Proof. $[\![C_1]\!] \cap [\![C_2]\!] = \bigcup_{c_1,c_2} [\![c_1]\!] \cap [\![c_2]\!]$ (distributivity), and each pair lands in the result: directly, or merged by `Clause::union` when two pairs produce the same var-set key — exact by Lemma 5. Dropped (`None`) pairs contribute nothing by Lemma 14. Invariant: the result is built without the top-collapsing constructor, but a top clause $\langle \emptyset \mid \top \rangle$ can arise only from a pair of top clauses, and a top clause is alone in its operand (invariant), so both operands were top and the result is the lone top clause. An empty product yields empty Clauses, i.e. $\bot$ at the TType level — semantic iff each pair's bottom was.
impl: `TTypeInter.sk / Clauses::inter`

□

4.5 The instance layer: TVar constraints, TInter, speculative returns

`inter_` has five early arms before the clause product; we audit each.

(i) Two unbound bare variables. Returns the unexpanded intersection $\mathsf{TInter}\{\alpha, \beta\}$, whose interpretation *is* $[\![\alpha]\!] \cap [\![\beta]\!]$ (the class comment's $\sqcap$-interpretation; its `expand` recomputes the intersection under the then-current state, covered by **L-EXP**). Containment is definitional; never $\bot$.
impl: `TTypeInter.sk / TInter (interpretation comment), TType::unexpandedInter`

(ii)/(iii) One bare variable, unbound or open-for-field. Returns α with the other operand attached as a constraint: `TVar::setConstraint` is a *functional* update — `TVar` is frozen, the method returns a new `TVar` value, and constraints are occurrence-level, not shared state (registry row SETCONSTRAINT-FUN, deep dive E) — so no other occurrence of α changes meaning. For the returned occurrence, adopt the natural sharpening of the main paper's TVar semantics: define $[\![\alpha^{cs}]\!]_\rho \triangleq [\![\alpha^{\emptyset}]\!]_\rho \cap \bigcap_{c \in cs} [\![c]\!]_\rho$, matching `Subst::get`, which intersects the binding's expansion with the occurrence's constraints at every read. Then $[\![\alpha^{cs}]\!] \cap [\![t]\!] = [\![\alpha^{cs \cup \{t\}}]\!]$ *exactly*, at every state — provided the get-time intersection itself over-approximates, which is L-INTER again, applied at read time; the circularity is benign (an inductive appeal at the strictly later read), but since the main paper states TVar semantics only as an upper bound, we record the sharpened definition as a named obligation for the TVar deep dive (Obligation 18). Never $\bot$.
impl: `TTypeInter.sk / inter_ (setConstraint arms); TType.sk / TVar::setConstraint; TTypeSubst.sk / Subst::get (constraint loop)`

(iv) In-flight cycle. If the pair is already being intersected (`tenv.seenInter`, an in-flight *set*, distinct from the memo cache), return the unexpanded intersection — same justification as (i); never $\bot$.

(v) Speculative boxed-primitive returns. When one side expands to a single primitive kind and the other to a single object type, the code checks $\mathsf{isSub}(\mathsf{Box}(k), \mathsf{object})$ and on success returns the *primitive operand unchanged* (e.g. `number & {toFixed: ...} = number`). Returning either operand always over-approximates the intersection, so these arms are sound regardless of the subtype verdict, and they never produce $\bot$. (They do evolve the state through `isSub`'s growth; the code comments note only concrete types are involved, and failure paths restore — the clone/restore discipline of the main paper.)
impl: `TTypeInter.sk / inter_ (primKind/getObject arms)`

Expansion drops nothing. The clause product runs on `expandToClauses(subst, tenv, Covariant())`, which discards the first component of `expandToVectors`. We verified that component is populated *only* when `expandTVars = None()` (variables pushed back unexpanded); with a variance supplied, every instance — `TSInst` (via the symbolically-cached expansion), `TInter`, `TArrayLit`, `TVar` — is rewritten into clauses, so the discarded vector is empty and no members are lost. A top expansion short-circuits to $\top$, exact for intersection's identity.
impl: `TType.sk / expandToVectors, expandToClauses`

4.6 The bottom inventory

Every site where intersection can produce $\bot$ (`None` or an empty structure), with its verdict. "Semantic" means the empty result implies empty semantic intersection; "derived" means the verdict is inherited from the sites it aggregates.

Site	When $\bot$	Nature of $\bot$
`PrimLiterals::inter`	spelling sets disjoint	semantic iff L-LIT(ii); spurious for the string kind (Finding 19)
`TPrimValue::inter`	kind mismatch	semantic (A1); unreachable from `STypes::inter`
`STypes::inter`, no shared tag survives	all per-tag results empty or unmatched	semantic (A1), derived
`STypes::inter`, empty shared tag	a shared tag's per-tag intersection is empty	semantic (A1); dropped, not zeroed
`TFields::inter`, brand	Some(true) vs Some(false)	semantic
`TFields::inter`, call/new	`TFuns::inter` fails both `isSub` directions	heuristic (Finding 13)
`TFields::interIsArray`	brand conflict at R-IsArray	semantic
`TObject::inter`	every branch pair `None`	derived
`Clause::inter` / `Clauses::inter`	STypes bottom / empty product	derived
`TInter::map`	a mapped component is $\bot$	derived (client utility, not on the `inter` path)
`inter_` short-circuit	an operand is $\bot$	semantic

Notably, *fields and indexes never produce* $\bot$: two named fields with disjoint gets intersect to a field whose get is $\bot$ — a branch that no object with that key present can inhabit, but the branch (and, if the field is optional, its members lacking the key) survives. The implementation's own `overlaps` comment documents this deliberately ("a shared field with non-overlapping types" is listed as a TODO for *warning* precision, not as a bottom). So the only unsound bottoms are the two Findings plus the conditional literal stratum — but each sits on a hot refinement path.

5 The Caches: seenUnion and seenInter

Both caches live in the substitution state, keyed by the operand pair, while the substitution evolves between write and reuse. The question is whether a cached result can become an *under*-approximation after evolution (unsoundness), as opposed to merely stale precision.

Theorem 16 (seenUnion Reuse is Unconditionally Sound)**.** *A* ***`seenUnion`*** *entry reused at any later state* ρ' *still satisfies the L-UNION containment at* ρ'*.*

Proof. By the proof of Theorem 8, `union_` never reads ρ: it is a pure function of the operands' syntax, and the containment proved is pointwise in ρ — the result type's members are (unions of) the very same syntactic components, so $[\![t_1]\!]_{\rho'} \cup [\![t_2]\!]_{\rho'} \subseteq [\![\mathsf{union}(t_1, t_2)]\!]_{\rho'}$ holds for every ρ' with one proof. The cached value equals the recomputation verbatim, so reuse is the identity. (This is the code comment's "TVars pass through unchanged" argument, made precise: state-independence of the computation plus state-uniformity of the theorem.) `restore` additionally rolls the cache back together with the bindings, which is needed only for precision hygiene, not soundness.

impl: `TTypeUnion.sk / TType::union (cache and comment); TTypeSubst.sk / SubstState.seenUnion, restore`

□

5.1 seenInter: the sealing argument and its freeze exception

`seenInter` is different in kind: `inter_` both reads and evolves the state. The implementation's justification (the comment on `TType::inter`) is a *sealing* argument:

1. Bare-variable operands bypass the cache entirely (the `hasTVar` guard) — exactly the arms of Section 4.5 that touch bindings without sealing.

2. Every variable a cached computation *did* read was read through `expandToClauses(..., Covariant())`, i.e. `Subst::get` covariantly, which seals it (SVOpen → SVClosed; fresh → SVClosed($\top$)); a sealed binding never changes again (main paper, Lemma "Determined Candidates Are Never Widened"(iii), which holds for all identifiers), so the expansion the cache baked in equals the expansion any later state would produce.

3. Named-type expansions consulted along the way (`seenExpansion`) are symbolic in their type arguments — the cached body mentions the argument *variables*, whose bindings are re-resolved at every use — so they cannot freeze a binding either (the argument documented at `SubstState.seenExpansion`).

4. `restore` rolls back `seenInter` together with the bindings, so failure-path exploration cannot leak entries into a state they were not computed against; and a `clone` shares the maps by value (immutable), so a clone's writes never reach the parent.

This argument is correct as far as it goes, but it has one exception, which we found by instruction-level audit of `Subst::get`: under `freezeCandidates`, a covariant read of an SVOpen *candidate* returns the value *without sealing*. An `inter` executed inside a constraint-admissibility check (the freeze window of `growTVars` — reachable, since admissibility calls `isSub`, whose validation intersects variable constraints through `Subst::get`) can therefore cache a result derived from a binding that remains growable after the freeze is popped; the freeze flag is deliberately outside `SubstState`, so no restore removes the entry. If the candidate later grows (a contravariant union enlarging its denotation), the cached intersection under-approximates the new $[\![t_1]\!] \cap [\![t_2]\!]$.

Obligation 17 (C-FREEZE-CACHE: seenInter Entries Written Under Freeze)**.** *The mechanism is* demonstrably active*: `TType::inter` is reached inside freeze windows with unsealed candidates (via `Subst::get`'s constraint-application loop), `seenInter` entries are written there, and an end-to-end order-dependent unsound acceptance exercising this path exists — shared with the sibling `subst.seenSub` memo (deep dive E's MR-1,* VIOLATED*; issue draft `issues/freeze-window-stale-memo-caches.md`). In particular, two otherwise-tempting deflections are false: admissibility checks do* not *save/restore around themselves on success paths, and freeze-window operand pairs do* not *always carry a bare variable. What remains open is* attribution *only: no witness isolates `seenInter`'s contribution from `seenSub`'s (every construction that reused a stale `seenInter` entry also reused a stale `seenSub` entry). Discharge the isolated statement, or apply the enforcing fix, which covers both memos: key `seenInter` by substitution version (as `tenv.seenSub` already is, after #446 — `subst.seenSub` itself is* not *version-keyed and shares this exposure), or skip cache* writes *while frozen.*
impl: `TTypeSubst.sk / Subst::get (freeze arm), Subst.freezeCandidates; TTypeInter.sk / TType::inter (hasTVar guard, re-entrant arm)`

Obligation 18 (TVAR-SEM: Sharpened Variable Denotation)**.** *Adopt (in the TVar/substitution deep dive) the definition* $[\![\alpha^{cs}]\!]_\rho = [\![\alpha^{\emptyset}]\!]_\rho \cap \bigcap_c [\![c]\!]_\rho$ *and verify it against every consumer of* `Subst::get/peek`*; Section 4.5(ii)'s exactness claim for constraint attachment is conditional on it. The main paper's one-sided inequality is implied by it.*

Two further hygiene points, verified benign: the re-entrant arm of `TType::inter` (a recursive call cached the pair first) returns the *already-cached* result rather than the just-computed one, for consistency with intermediate consumers — soundness is unaffected since both are results of `inter_` on the same pair; and the `ty1 == ty2` fast path returns `ty1`, exact for idempotent operands.

6 Empirical Witnesses

All probes were run against the pinned binary (`target/host/dev/skjs`, built from `cf5a6bc7`) via `make check-file-in-docker`. "Accepted" means zero diagnostics.

6.1 Probe 2: signature bottom (Finding 13)

```
function p2(k: (s: string) => void, g: (n: number) => void): void {
  if (k === g) {
    const z: never = k;   // ACCEPTED (silently)
    console.log(z);
  }
}
```

Runtime witness: `const h = (x: unknown) => {}` inhabits both parameter types; `p2(h, h)` takes the branch — and the call `p2(h, h)` is itself accepted by the checker, so a single accepted program demonstrates the unsoundness end-to-end (issue draft `issues/tfun-inter-incomparable-not-disjoint.md`). tsc flags the comparison itself with TS2367; either way tsc rejects the program.

6.2 Probe 3: literal spellings (Finding 19)

Finding 19 (Spelling-Sensitive Disjointness)**.**

```
function p3(a: "\x61", b: "a"): void {
  if (a === b) {          // warning: "comparison is always false
    const z: never = a;   //   because the operand types do not
    console.log(z);       //   overlap" -- and z: never ACCEPTED
  }
}
```

`"\x61" === "a"` is true *at runtime (both denote `"a"`), so both the warning and the narrowing are wrong. This is not a new root cause: it is the* string*-kind* ***L-LIT****(ii) violation (escape sequences stored undecoded, issue draft `issues/string-literal-escapes-undecoded.md`) propagating through Lemma 10 into `PrimLiterals::inter`'s bottom and thence into `overlaps` and the branch logic — L-INTER's* disjointness *reading breaking on it. Numeric spellings do not trigger it: the parser normalizes numeric literals, so `0.0` and `0` share one lexeme.*

6.3 Probe 4: type-level intersections

```
type C = ("a" | 0) & ("a" | 1);
function p6(): void { const c: C = "a"; }        // ACCEPTED: C = "a"
type F = ((s: string) => void) & ((n: number) => void);
function p8(f: F): void { const z: never = f; }  // ACCEPTED: F = never
```

The STypes assembly is exact: `C = "a"` (the empty `num` tag is dropped, not zeroed), matching tsc. The call/new stratum still collapses: `F` is `never` where tsc reads a callable overload — `p8`'s acceptance is the incompleteness dual of Finding 13.

6.4 Failed proof attempts, recorded

For completeness, the containments we attempted and could not close, each of which became a Finding or Obligation rather than a theorem: (1) call/new-level $\bot$-implies-disjoint — refuted (Finding 13); (2) literal-level $\bot$-implies-disjoint without the string-kind L-LIT(ii) — refuted (Finding 19); (4) unconditional `seenInter` reuse — the sealing argument's freeze gap resisted proof and is Obligation 17; (5) exactness of constraint attachment from the main paper's one-sided TVar axiom alone — needs the sharpened denotation, Obligation 18. We found *no* defect in: any union stratum, the field/index/brand strata of intersection, the instance-layer arms, the clause product, or the in-flight cycle handling.

7 Status and Ledger Edits

7.1 Status by stratum

Stratum	**Result**
Union: literals, prims, units	Lemmas 1, 2 (exact)
Union: object branch sets	Lemma 3 (exact)
Union: STypes / Clause / Clauses / TInsts	Lemmas 4–7, Theorem 8 (exact)
Union: branch-merge client (`TFields::union`)	Lemma 9 (over-approx.)
`seenUnion` cache	Theorem 16 (unconditional)
Inter: units, tag dropping, no-shared-tag $\bot$	Lemma 11, A1
Inter: literal sets	Lemma 10, conditional on L-LIT(ii); string-kind escapes spurious (Finding 19)
Inter: STypes assembly	exact — empty shared tags dropped, $\bot$ iff all drop
Inter: named fields, indexes, brand, access	Section 4.3 (brand $\bot$ semantic; given sub-level L-INTER)
Inter: call/new	signature-failure branch drop (Finding 13, pinned probe)
Inter: Clause/Clauses product	Lemmas 14, 15 (derived)

Stratum	Result
Inter: instance layer (constraints, TInter, speculative returns)	Section 4.5; never $\bot$, conditional on Obligation 18
`seenInter` cache	sealing argument + freeze gap (Obligation 17)
Cloned-subst reuse of results	clone-stability, via the fork-state form I_f of the containments audited here

7.2 Stratum outcomes

L-UNION. PROVED (this document, Theorem 8 with Lemmas 1–9 and Theorem 16 for the cache). Note field: "exact for membership semantics at every stratum on the $\mathsf{union}(t_1, t_2)$ path; the branch-merge client `TFields::union` is over-approximating only. Signature bodies built by `TFun::union` are outside membership semantics and remain part of the subtyping surface."

L-INTER. Established stratum by stratum:

- **L-INTER-a** (units, tags, fields, indexes, brand, clause product, instance layer): PROVED (this document, Sections 4.1–4.5); the instance layer conditionally on **TVAR-SEM** (Obligation 18).
- **L-INTER-b** (literal sets): VIOLATED for the *string* kind — inherits **L-LIT**(ii) undecoded escapes; witness pinned (Finding 19). Numeric spellings are normalized at parse time, so that half is PROVED via Lemma 10.
- **L-INTER-c** (STypes assembly): PROVED — empty shared tags are dropped, $\bot$ only when all drop (Section 4.2).
- **L-INTER-d** (call/new): VIOLATED — heuristic branch drop on incomparable signatures, witness pinned (Finding 13). Fix: never convert `TFuns::inter` failure into `None`; keep a widened or overloaded signature.
- **L-INTER-e** (`seenInter` reuse): OPEN — Obligation 17 (C-FREEZE-CACHE).

Every theorem of the main paper that cites **L-INTER** — the branch-drop steps of R-Eq, R-EqField, R-Inst, R-IsArray, and the equality-narrowing intersection itself — is affected by b–d exactly where its operands reach a violated stratum; the R-IsArray drops are unaffected (their bottoms are the brand and no-shared-tag cases, both semantic — except when the intersected array branch pair hits the call/new arm, which `ReadonlyArray` does not). As with the main paper's other VIOLATED entries, the violations are local to leaf strata: b–d are code fixes that change no statement or proof in this document, after which L-INTER-a–d are all PROVED and only the OPEN obligation C-FREEZE-CACHE remains here. Each of the three witnesses should be wired as a pinned test per the main paper's maintenance discipline (probes 1–4, Section 6).

Registry. The authoritative, current statuses are maintained in the registry (`docs/ledger.yaml`), rendered as the main paper's Assumptions and Proof Obligations tables; this paper is cited there as evidence. The registry rows it supports:

Row	Statement	Status
L-UNION	`union` over-approximates set union (all levels).	PROVED
L-INTER	`inter` over-approximates set intersection (the containment clause).	PROVED
L-INTER-BOT	A $\bot$ result from `inter` implies semantic disjointness.	VIOLATED
C-FREEZE-CACHE	Isolated attribution of `seenInter` freeze-window staleness (mechanism active; shared witness with MR-1).	OPEN
TVAR-SEM	Sharpened variable denotation (get-time constraint intersection as definition), verified against every consumer.	OPEN

Imports (review contract). The named statements this paper assumes from elsewhere. A reviewer treats each as an axiom, checking citations against the registry's canonical statement; no other paper's internals are in scope:

Row	Canonical statement	Owner	Status
L-EXP	Expansion preserves or enlarges semantics (modes `None`/Covariant); residual instances keep theirs.	C	PROVED
L-LIT	Literal spellings: (i) δ is a function; (ii) injectivity modulo $\approx$ after normalization; (iii) falsiness-exactness of the lexeme test.	G	VIOLATED
MR-1	No `subst.seenSub` memo recorded under `freezeCandidates` may outlive growth of an open candidate readable from its left side.	E	VIOLATED
SETCONSTRAINT-FUN	`TVar::setConstraint` is a functional update on a frozen value: constraints are occurrence-level, not shared state.	E	PROVED

8 Implementation Anchor Index

Pinned to `cf5a6bc7`.

This document	Implementation
union top level, cache	`TTypeUnion.sk / TType::union, union_`
literal-set ops	`PrimLiterals.sk / union, inter, diff`
primitive merges	`TTypeUnion.sk / TPrimValue::union`; `TTypeInter.sk / TPrimValue::inter`
STypes assembly	`TTypeUnion.sk / STypes::union`; `TTypeInter.sk / STypes::inter`
map combinators	`InternalKeyedArrayMap.sk / mapUnion, mapInterOption, mergeOption, mergeOpt`
object branch sets	`TTypeUnion.sk / TObject::union`; `TTypeInter.sk / TObject::inter`
branch pair	`TTypeInter.sk / TFields::inter, interNamedFields, TIndexes::inter`
branch merge (reads)	`TType.sk / maybeGetType, resolveLiteralAccess, maybeGetSymbol, maybeGetCall/New`
fields	`TTypeInter.sk / TField::inter, interIndex`; `TTypeUnion.sk / TField::union, unionIndex`
access lattice	`TType.sk / FieldAccess::combine`
call/new	`TTypeInter.sk / TFun::inter, TFuns::inter`; `TTypeUnion.sk / TFun::union`
brand	`TTypeInter.sk / TFields::inter (isArray), interIsArray`
clause strata	`TTypeUnion.sk` / `TTypeInter.sk` / `Clause::*`, `Clauses::*`; `TType.sk / Clauses::create`
instance layer	`TTypeInter.sk / TInter, inter_ (var arms)`; `TType.sk / TVar::setConstraint`
expansion	`TType.sk / expandToVectors, expandToClauses`
caches	`TTypeSubst.sk / SubstState.seenUnion, seenInter, seenExpansion, restore; Subst::clone, get, freezeCandidates`
overlap warning	`TTypeInter.sk / SType::overlaps`
refinement consumers	`TypeCheckRefine.sk / evalVarEquality, evalFieldEqualityType, narrowField, evalIsArray, evalInstanceof`